BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION SYSTEMS

Compiler Construction (CS F363)
II Semester 2025-26
Compiler Project
Coding Details
(March 02, 2026)

Group Number
10

1. Team Members Names and IDs

ID: 2022B5A70783P	Name: Shiv Tiwari
ID: 2022B5A70724P	Name: Samyek Jain
ID: 2022B3A70551P	Name: Ayush Jain
ID: 2022B4A71282P	Name: Nachiket Garg
ID: 2022B5A71326P	Name: Mayukh Khetan
ID: 2022B5A70763P	Name: Shubham Mishra

2. Mention the names of the Submitted files:

1. driver.c	7. parserDef.h	13. grammar.txt
2. lexer.c	8. makefile	14. testcase1.txt
3. lexer.h	9. t5.txt	15. testcase2.txt
4. lexerDef.h	10. coding_details.pdf	16. testcase3.txt
5. parser.c	11. commentremoval_t2.txt	17. testcase4.txt
6. parser.h	12. t6.txt	18. testcase5.txt
19. testcase6.txt	20. t1.txt	21. t2.txt
22. t3.txt	23. t4.txt	

3. Total number of submitted files (including copy the pdf file of this coding details pro forma) : 13 (All files should be in ONE folder named as Group_#)

4. Have you compressed the folder as specified in the submission guidelines? (yes/no): yes

5. **Lexer Details:**

[A]. Technique used for pattern matching: Explicit DFA simulation coded directly in C. Each character class and token type is handled via a hand-written sequence of conditional checks and character-by-character advances/retracts through the twin buffer, effectively encoding the DFA transitions as an imperative control flow.

[B]. Keyword Handling Technique: Linear search through a static keyword table. The function lookupKeyword() iterates over the {keyword, TokenType} pairs and uses strcmp() to match the lexeme. If no match is found, the token is classified as TK_FIELDID.

[C]. Hash function description, if used for keyword handling: NA

[D]. Have you used twin buffer? (yes/ no): yes

[E]. Error handling and reporting (yes/No): yes

[F]. Describe the errors handled by you:

1) Unknown symbol: isolated character that does not correspond to any valid token (e.g., single &, @, =, !, |, or standalone _)

2) Unknown pattern: incomplete or badly-formed multi-character token (e.g., &&, @@, <-, <--, incorrectly formatted real numbers or exponents).

3) Identifier too long: variable identifier exceeding 20 characters (ERR_ID_TOO_LONG)

4) Function identifier too long: function identifier exceeding 30 characters (ERR_FUNID_TOO_LONG). All reported errors include the corresponding line number and the problematic lexeme.

[G]. Data Structure Description for tokenInfo (in maximum two lines): tokenInfo is a structure containing the token category (TokenType), the lexeme string, and the source line number. It also includes a numeric value union (int/double) with a validity flag, along with an error type code and an associated error message buffer.

6. Parser Details:

[A]. High Level Data Structure Description (in maximum three lines each, avoid giving C definitions used):

i. grammar : The grammar is stored as a struct containing an array of GrammarRule (up to 140 rules), each rule holding its LHS NonTerminal, an array of GrammarSymbol on the RHS (up to 15 symbols), the count of RHS symbols, and a rule number. Each GrammarSymbol has a type (terminal/non-terminal) and a union of TokenType or NonTerminal.

ii. FIRST and FOLLOW sets: Stored in a FirstAndFollow struct that contains an array of FirstSet and an array of FollowSet (one per non-terminal). Each FirstSet holds the NonTerminal, an array of up to 45 terminal TokenTypes, a count, and a hasEpsilon flag. Each FollowSet holds the NonTerminal and an array of up to 45 terminal TokenTypes with a count.

iii. parse table: Stored as a struct table containing a 2D array of ParseTableEntry[MAX_NT][TABLE_TERMINALS] (75 non-terminals x 70 terminals). Each ParseTableEntry has an isValid flag, the rule number, and a pointer to the corresponding GrammarRule. Indexing is done directly by (NonTerminal enum, TokenType enum value cast to int), not via a separate "token index mapping" function.

iv. parse tree: (Describe the node structure also) The parseTree struct holds a pointer to the root parseTreeNode. Each node contains: a GrammarSymbol (type + terminal/non-terminal union), a tokenInfo (for leaf nodes), an array of up to 15 child pointers, childCount, a parent pointer, an isLeaf flag, and a visited flag. Internal nodes store the non-terminal; leaf nodes store the matched token.

v. Any other (specify and describe): An explicit stack is used to implement the LL(1) predictive parsing algorithm. Each stack element stores a grammar symbol (terminal or non-terminal) along with a pointer to the corresponding parse tree node, enabling simultaneous syntactic analysis and incremental construction of the parse tree during rule expansion and terminal matching.

[B]. Parse tree

i. Constructed (yes/no): yes

ii. Printing as per the given format (yes/no): yes

iii. Describe the order you have adopted for printing the parse tree nodes (in maximum two lines): The parse tree is printed using an in-order (inorder) traversal implemented in printInorder(). For each internal (non-terminal) node, the first child is visited first, then the node itself is printed, then the remaining children are visited left to right.

[C]. Grammar and Computation of First and Follow Sets

i. Data structure for original grammar rules: Array of GrammarRule structs (grammar.rules[], max 140). Each GrammarRule stores the LHS NonTerminal, RHS as GrammarSymbol array (max 15), rhsCount, and ruleNumber.

ii. FIRST and FOLLOW sets computation automated (yes /no): yes

iii. Name the functions (if automated) for computation of First and Follow sets: computeFirstAndFollowSets() (driver), computeFirstOne() (per non-terminal FIRST), computeFollowOne() (per non-terminal FOLLOW) - all in parser.c

iv. If computed First and Follow sets manually and represented in file/function (name that): NA. FIRST and FOLLOW sets are computed automatically.

[D]. Error Handling

v. Attempted (yes/ no): yes

vi. Describe the types of errors handled:

1) Unexpected token at end of input

2) Token mismatch: actual token does not match expected terminal on stack top

3) Invalid token with no parse table entry (empty cell): triggers panic-mode error recovery by skipping tokens until a synchronising token usable by some item on the parse stack is found.

4) lexical TK_ERROR tokens are handled in nextMeaningfulToken() by reporting them and continuing (they are not fed into the parse loop)

7. Compilation Details:

[A]. Makefile works (yes/no): yes
[B]. Code Compiles (yes/ no): yes
[C]. Mention the .c files that do not compile: NA
[D]. Any specific function that does not compile: NA
[E]. Ensured the compatibility of your code with the specified gcc version (yes/no): yes

8. Driver Details: Does it take care of the options specified earlier(yes/no): yes
9. Execution
   [A]. status (describe in maximum 2 lines): Both lexer and parser modules are fully functional. The compiler correctly tokenises input, computes FIRST/FOLLOW sets automatically, builds the LL(1) parse table, performs parsing with panic-mode error recovery, constructs the parse tree, and option 3 is safe only when the program is run with the second CLI argument (parse tree output file); otherwise fopen(NULL,"w") can crash.

   [B]. Gives segmentation fault with any of the test cases (1-6) uploaded on the course page. If yes, specify the testcase file name: No

10. Specify the language features your lexer or parser is not able to handle (in maximum one line) : We have run tests on multiple cases and have not found any features where our lexer or parser is not working.

11. Are you availing the lifeline (Yes/No): No

12. Declaration: We, Shiv, Samyek, Ayush, Nachiket, Mayukh, and Shubham declare that we have put our genuine efforts in creating the compiler project code and have submitted the code developed only by us. We have not copied any piece of code from any source. If our code is found plagiarized in any form or degree, we understand that a disciplinary action as per the institute rules will be taken against all of us in our team and we will accept the penalty as decided by the department of Computer Science and Information Systems, BITS, Pilani.

Your names and IDs
Name: Shiv Tiwari ID: 2022B5A70783P
Name: Samyek Jain ID: 2022B5A70724P
Name: Ayush Jain ID: 2022B3A70551P
Name: Nachiket Garg ID: 2022B4A71282P
Name: Mayukh Khetan ID: 2022B5A71326P
Name: Shubham Mishra ID: 2022B5A70763P
Date: 02/03/2026

Not to exceed 3 pages.